Software Architecture and Design Specification (SAD)

Vehicle Parking System (VPS)

Course: Software Engineering (UE24CS341A)
Institution: PES University
Document Version: 0.1 (Proposed Draft)
Status: Draft / Pending Review

Document Revision History

Version	Date	Author(s)	Description / Key Changes
0.1	TBD	VPS Engineering Team	Initial proposed draft & proposed baseline architecture specification

Document Approvals

Role	Name / Title	Signature / Approval Status	Date
Academic Evaluator	Course Faculty (UE24CS341A)	Pending Review	TBD
System Architect	Lead Software Engineer	Pending Review	TBD
Quality Assurance	QA Lead	Pending Review	TBD

---

1. Introduction

1.1 Purpose

This Software Architecture and Design Specification (SAD) provides a comprehensive description of the architectural structure, design decisions, component interfaces, data models, and operational flows for the **Vehicle Parking System (VPS)**. It bridges the system requirements articulated in the *Software Requirements Specification (SRS)* with the actual software implementation, serving as the primary design reference for developers, testers, and evaluators.

1.2 Scope

The Vehicle Parking System (VPS) is a proposed enterprise web application designed to manage multi-lot parking facilities, automate check-in/check-out processes, dynamically assign slots based on vehicle types, enforce flexible pricing policies, and maintain auditability.

- **Baseline Functional Scope (FR-01 to FR-14):** Core operational capabilities implemented in the production baseline, including multi-tier user authentication, vehicle entry registration, dynamic slot assignment algorithm, checkout automated bill generation, pricing tier management, and real-time dashboard monitoring.
- **Provisional Team Backlog Scope (FR-15 to FR-30):** Proposed architectural provisions designed to support advance slot reservations, subscription/pass management, multi-lot management, and automated notifications pending verification against SRS version discrepancies.
- **Optional Future Enhancements (Non-Committed):** Unapproved enterprise features such as ANPR camera integration, physical gate hardware barriers, automated surge pricing, overstay fine enforcement, EV charging metrics, offline entry sync queues, and multi-region database failover are out of scope for the core baseline.

1.3 Target Audience

This document is structured for the following stakeholders:

- **Software Engineers & Backend Developers:** To construct components, DB schemas, and RESTful endpoints consistent with the architectural patterns.
- **Quality Assurance Engineers:** To design integration, performance, and security test suites based on structural contracts and sequence flows.
- **Academic Evaluators & Reviewers:** To verify compliance with software engineering methodologies under course UE24CS341A.

- **System Administrators & DevOps:** To deploy, manage, and scale the modular monolith on containerized infrastructures.

1.4 Terminology & Definitions

- **Modular Monolith:** An architectural design style where the application code is organized into loosely coupled, independent modules within a single unified deployment unit and shared database schema.
- **Slot Assignment Algorithm:** A deterministic spatial allocation routine that maps an incoming vehicle to the closest available slot matching its exact size category (e.g., Two-Wheeler, Compact, SUV, Electric Vehicle).
- **Check-In / Check-Out:** The operational lifecycle transitions representing vehicle entry (issuing a session record) and vehicle exit (calculating duration, generating a bill, and releasing the slot).
- **STRIDE:** A threat modeling framework representing Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, and Elevation of Privilege.

---

2. Document Overview

2.1 How to Use This Document

- **Architectural Understanding:** Read **Section 3** to understand high-level structural patterns, global technical stack selection, risk evaluations, requirement traceability, and system security bounds.
- **Implementation & Coding:** Refer to **Section 4** for data models (ER diagram structures), message sequence diagrams, REST API endpoints, exact HTTP payload structures, error taxonomy, and UX design guidelines.
- **Maintenance & Next Steps:** Consult **Section 4.6** and **Section 5** for open design issues, delta analysis between SRS versions, and technical references.

2.2 Related Documents

1. *Vehicle Parking System - Software Requirements Specification (SRS)*, Version 1.1.
2. *UE24CS341A Team Handbook & Project Guidelines*, PES University.
3. *IEEE Std 1016-2009: IEEE Standard for Information Technology—Systems Design—Software Design Descriptions*.

3. System Architecture

3.1 Architectural Goals & Constraints

Architectural Goals

1. **Maintainability & Low Complexity:** Avoid microservice overhead (e.g., distributed transactions, service discovery) while enforcing module boundary isolation.
2. **Strict Operational Integrity:** Prevent over-allocation of physical parking spaces under high concurrent entry requests (ACID transactional guarantees).
3. **Extensibility:** Provide abstract interfaces allowing seamless future integration of physical hardware (IoT barriers, ANPR cameras) and payment gateways without modifying core business rules.
4. **Auditability & Non-Repudiation:** Maintain an immutable record of all operational state changes (entry, exit, override, payment, and user actions).

Constraints

- **Deployment Environment:** Single-host containerized or virtualized instance running Python 3.11+ and PostgreSQL 15+.
- **Framework Baseline:** Django 4.2+ with Django REST Framework (DRF) for the backend REST APIs, and React 18+ with Vite for the frontend Single Page Application (SPA).
- **Course Boundaries:** Execution within academic resource allocations, focusing on logical backend interfaces rather than physical gate microcontrollers.

3.2 Stakeholders & Key Concerns

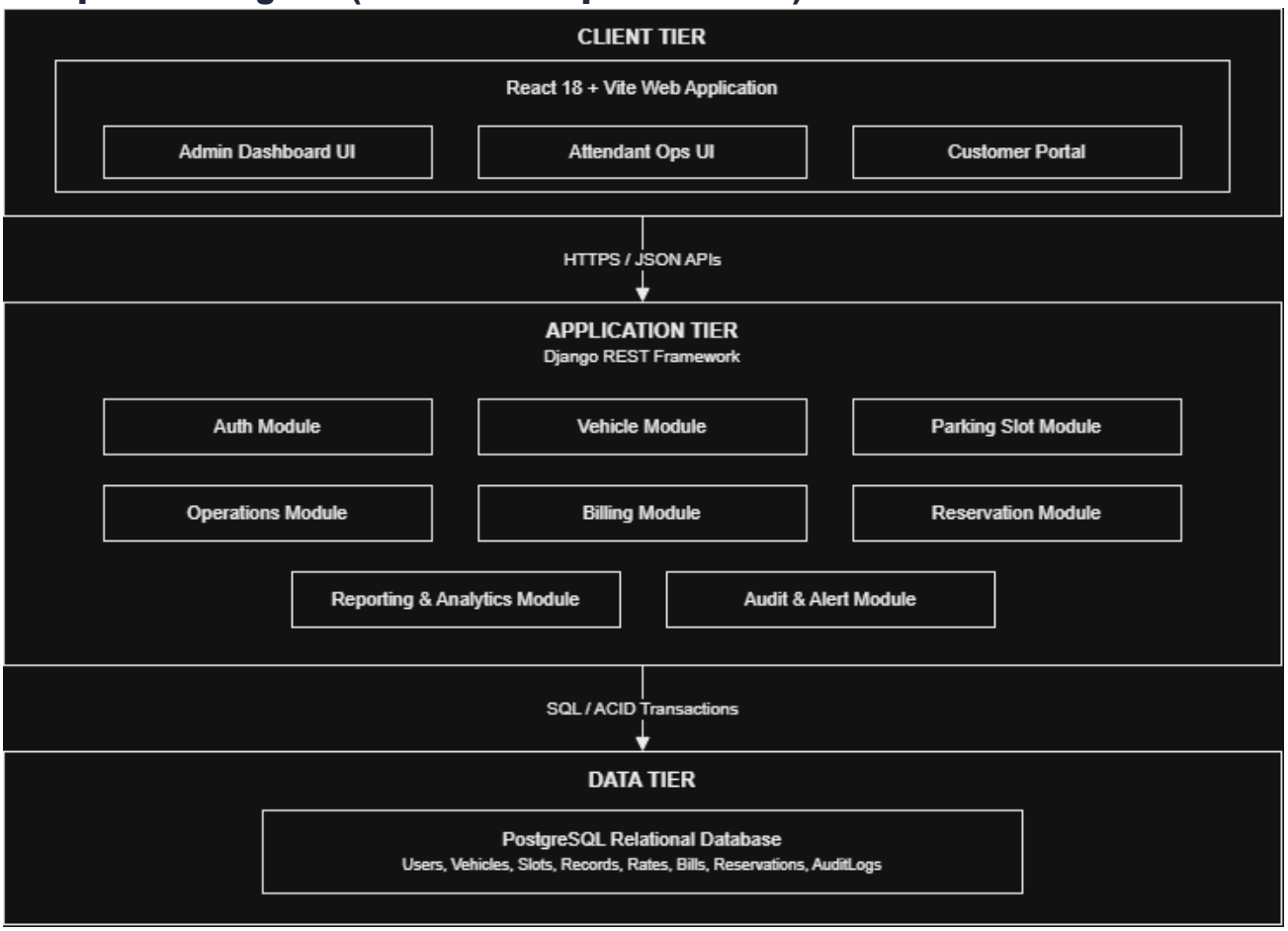
Stakeholder	Key Concerns	Architectural Resolution
System Administrator	User account provisioning, system health monitoring, global configuration management, audit trail verification.	Dynamic RBAC implementation, Django Admin module integration, centralized logging, audit log tables.
Parking Attendant / Operator	Fast check-in/out execution (<2 sec processing time), real-time slot state visibility, manual overrides for faulty gates/cards.	React SPA dashboard with auto-refresh/WebSockets, optimized index queries, atomic transactional endpoints.

Stakeholder	Key Concerns	Architectural Resolution
End User / Driver	Transparent billing, availability visibility, pass/reservation capabilities.	Customer portal API endpoints, clear invoice structures, self-service slot booking interfaces.
Software Architect	Code maintainability, boundary separation, scalability, security threat mitigation.	Modular Monolith pattern, strict Django app isolation, STRIDE-driven security controls, unified data schema.

3.3 Component Architecture Overview

The Vehicle Parking System is designed as a **Modular Monolith** deployed via a **Layered Web Architecture**. The system UI is completely decoupled from backend logic, communicating exclusively via RESTful JSON endpoints secured by JWT authentication.

Component Diagram (Structural Representation)



3.4 Component Descriptions

1. **User Interface Component (React SPA):** Renders dynamic operational dashboards, provides real-time visual feedback on slot state maps, captures input for entry/exit processing, and stores JWT tokens in secure client storage.
2. **Authentication & Authorization Module:** Manages user identity verification via JWT tokens, handles password hashing using Argon2/PBKDF2, and enforces role-based access control (Admin, Attendant, Customer).
3. **Vehicle Management Module:** Validates license plate formats, tracks vehicle categories (Compact, SUV, Two-Wheeler, EV), and maintains historical access associations per vehicle.

4. **Parking Slot Allocation Engine:** Executes the slot selection logic based on physical size matching, proximity prioritization, and dynamic occupancy states (`AVAILABLE`, `OCCUPIED`, `RESERVED`, `MAINTENANCE`).
5. **Parking Operations Module:** Orchestrates end-to-end check-in and check-out workflows, links vehicles to assigned slots via active `ParkingRecord` entities, and enforces atomic transactional state updates.
6. **Billing & Pricing Module:** Implements tariff evaluation rules (hourly rates, vehicle type multipliers, overnight surcharges), computes total billable amounts, and logs payment status transitions.
7. **Reservation Module (Provisional Scope):** Proposed module to manage advance slot holds for authenticated Customers and Admins, handle expiration timers for unfulfilled bookings, and mark slots as reserved prior to customer arrival.
8. **Reporting & Analytics Module:** Generates aggregated statistical metrics regarding lot usage trends, peak occupancy hours, daily revenue metrics, and attendant throughput.
9. **Audit & Notification Module:** Proposed module to record security-critical actions (manual overrides, price changes, system configuration updates) and dispatch user notifications.
10. **PostgreSQL Relational Database:** Provides persistent storage, enforces referential integrity constraints, executes relational joins, and guarantees ACID properties during simultaneous entry/exit updates.

3.5 Chosen Architecture Pattern & Rationale

Selected Architecture: Layered Web Architecture / Modular Monolith

The architecture combines a **Layered Monolith** on the backend with a fully separated **Single Page Application (SPA)** client.

Presentation Layer (React SPA / REST Controllers)
Application / Business Logic Layer (Services & Enforcers)
Persistence Layer (Django ORM Models)
Database Layer (PostgreSQL Engine)

Rationale for Selection

- **Operational Simplicity vs Microservices:** Microservices introduce operational complexity (network latency, distributed transaction orchestration, complex deployment pipelines) that is unnecessary for single-facility or centralized multi-lot parking management.

- **Strong Domain Boundaries:** A Modular Monolith allows clean code separation by organizing Django apps by domain (`users`, `vehicles`, `slots`, `operations`, `billing`). If transaction volume scales significantly in future iterations, these modules can be converted into standalone microservices.
- **ACID Compliance:** Parking slot allocation requires strict, atomic synchronization to prevent two attendants from issuing the same slot simultaneously. A centralized database engine executing `SELECT ... FOR UPDATE` locks natively guarantees zero double-bookings.

3.6 Technology Stack & Data Stores

Layer	Technology Selected	Version / Details	Architectural Justification
Frontend Framework	React	v18.x	Declarative component model, efficient DOM diffing for dynamic grid maps.
Frontend Build Tool	Vite	v5.x	Instant HMR (Hot Module Replacement) and optimized bundling capabilities.
UI Components	Tailwind CSS / Lucide	Latest	Rapid responsive dashboard styling and UI status icon rendering.
Backend Framework	Django / DRF	v4.2+ / v3.14+	Robust ORM, built-in security features (CSRF, SQLi protection), native REST suite.
Authentication	djangorestframework-simplejwt	v5.x	Stateless JWT-based authentication across API endpoints.
Database Engine	PostgreSQL	v15.x	Highly reliable ACID compliance, robust indexing, native JSONB support for audit trails.
API Architecture	RESTful JSON	Open API 3.0 Specs	Standardized, self-documenting HTTP interfaces.

3.7 Architectural Risks & Mitigations

Risk ID	Identified Risk	Impact	Risk Level	Proposed Architectural Mitigation Strategy
RSK-01	Race conditions during peak traffic causing double slot assignment.	High	High	Propose database-level row locks (`SELECT FOR UPDATE`) within transactional blocks during check-in execution.
RSK-02	Stale parking lot state on attendant dashboard during continuous operations.	Medium	Medium	Propose visual polling fallback intervals or WebSocket channels for real-time slot state propagation.
RSK-03	Loss of billing history data due to dynamic tariff rule updates.	High	High	Store applied rate snapshots directly within the immutable `Bill` record upon issuance rather than referencing dynamic tables.
RSK-04	Module tight-coupling preventing independent module updates.	Medium	Low	Enforce strict internal API service interfaces between Django apps; disallow direct cross-module model writes.
RSK-05	Unauthorized access to administrative configuration interfaces.	Critical	High	Propose strict RBAC middleware checking user roles on both API controllers and UI view routes.

3.8 Traceability Matrix to System Requirements

The table below maps the authoritative SRS v1.0 baseline requirements (FR-01 to FR-14) and provisional team backlog requirements (FR-15 to FR-30) to target architecture modules and proposed implementation status.

Req ID	Requirement Description	Architectural Module	Status
FR-01	User authentication with role-based access control (Admin, Attendant, Customer).	Auth Module	Proposed (SRS v1.0 Baseline)

Req ID	Requirement Description	Architectural Module	Status
FR-02	Secure password management and session handling via JWT.	Auth Module	Proposed (SRS v1.0 Baseline)
FR-03	Vehicle registration and plate lookup capabilities.	Vehicle Module	Proposed (SRS v1.0 Baseline)
FR-04	Parking slot creation, updates, and maintenance status tracking.	Parking Slot Module	Proposed (SRS v1.0 Baseline)
FR-05	Automatic slot allocation matching vehicle dimensions.	Slot Allocation Engine	Proposed (SRS v1.0 Baseline)
FR-06	Real-time visual display of parking lot grid availability.	Operations / UI Module	Proposed (SRS v1.0 Baseline)
FR-07	Vehicle entry processing and timestamp record generation.	Operations Module	Proposed (SRS v1.0 Baseline)
FR-08	Vehicle exit processing and session duration calculation.	Operations Module	Proposed (SRS v1.0 Baseline)
FR-09	Automated fee computation based on vehicle type and duration.	Billing Module	Proposed (SRS v1.0 Baseline)
FR-10	Receipt and invoice generation upon successful checkout.	Billing Module	Proposed (SRS v1.0 Baseline)
FR-11	Dynamic pricing tier configuration by system administrators.	Billing Module	Proposed (SRS v1.0 Baseline)
FR-12	System dashboard displaying active vehicles and real-time occupancy.	Reporting Module	Proposed (SRS v1.0 Baseline)
FR-13	Historical record search, filter, and export tools.	Operations / Reporting	Proposed (SRS v1.0 Baseline)
FR-14	Audit logging of privileged administrative overrides.	Audit Module	Proposed (SRS v1.0 Baseline)
FR-15	Advance parking slot reservation and hold scheduling.	Reservation Module	Provisional (Team Backlog)
FR-16	Reservation cancellation and buffer handling.	Reservation Module	Provisional (Team Backlog)
FR-17	Digital pass and recurring subscription management.	Billing / Vehicle	Provisional (Team Backlog)

Req ID	Requirement Description	Architectural Module	Status
FR-18	Gate barrier hardware integration (Out of Scope).	N/A (Future Enhancement)	Unapproved / Out of Scope
FR-19	ANPR camera feed processing (Out of Scope).	N/A (Future Enhancement)	Unapproved / Out of Scope
FR-20	Multi-lot management supporting separate geographical zones.	Slot / Operations	Provisional (Team Backlog)
FR-21	Dynamic surge pricing algorithms (Out of Scope).	N/A (Future Enhancement)	Unapproved / Out of Scope
FR-22	Overstay fine enforcement rules (Out of Scope).	N/A (Future Enhancement)	Unapproved / Out of Scope
FR-23	Automated notification dispatch (SMS/Email) for receipts and holds.	Notification Module	Provisional (Team Backlog)
FR-24	EV charging station metrics integration (Out of Scope).	N/A (Future Enhancement)	Unapproved / Out of Scope
FR-25	Customer mobile web app for self-check-in and payment handling.	UI Client	Provisional (Team Backlog)
FR-26	Comprehensive financial analytics and forecasting dashboard.	Reporting Engine	Provisional (Team Backlog)
FR-27	Custom role definition and dynamic permission mapping.	Auth Module	Provisional (Team Backlog)
FR-28	Dynamic multi-currency support and localization formatting.	Billing Module	Provisional (Team Backlog)
FR-29	Offline sync queues (Out of Scope).	N/A (Future Enhancement)	Unapproved / Out of Scope
FR-30	Multi-region database failover (Out of Scope).	N/A (Future Enhancement)	Unapproved / Out of Scope

3.9 Security Architecture

Threat Modeling (STRIDE Analysis)

STRIDE THREAT MODEL SUMMARY

Threat Category	Identified System Vulnerability	Mitigation Mechanism
Spoofing	Attendant impersonation via session hijacking.	Short-lived JWT access tokens with secure HTTPS transmission.
Tampering	Altering billing rate structures or calculated total fees.	Enforce administrative RBAC checks & immutable billing logs
Repudiation	Manual override performed without operator accountability.	Immutable `AuditLog` table with timestamp, User ID, and IP.
Information Disclosure	Exposing vehicle entry patterns or phone numbers to public APIs.	Obfuscate user PII; enforce strict REST permissions.
Denial of Service	Flooding `/check-in` endpoints to deplete database connections.	Rate-limiting throttles; DRF connection pool management.
Elevation of Privilege	Customer escalating privilege to alter parking slot statuses.	Centralized DRF permission classes (`IsAdminUser`).

Role-Based Access Control (RBAC) Matrix

System Module / Capability	Administrator	Parking Attendant	Customer	Anonymous
User Provisioning & Role Assign	Full Access	No Access	No Access	No Access
Configure Rates & Slot Structure	Full Access	Read Only	Read Only	No Access
Execute Check-In / Check-Out	Full Access	Full Access	No Access	No Access
View Live Parking Grid	Full Access	Full Access	Read Only	Read Only
Create Advance Reservation	Full Access	No Access	Create / View	No Access

System Module / Capability	Administrator	Parking Attendant	Customer	Anonymous
Access Financial & Audit Reports	Full Access	No Access	Self History	No Access

4. System Design

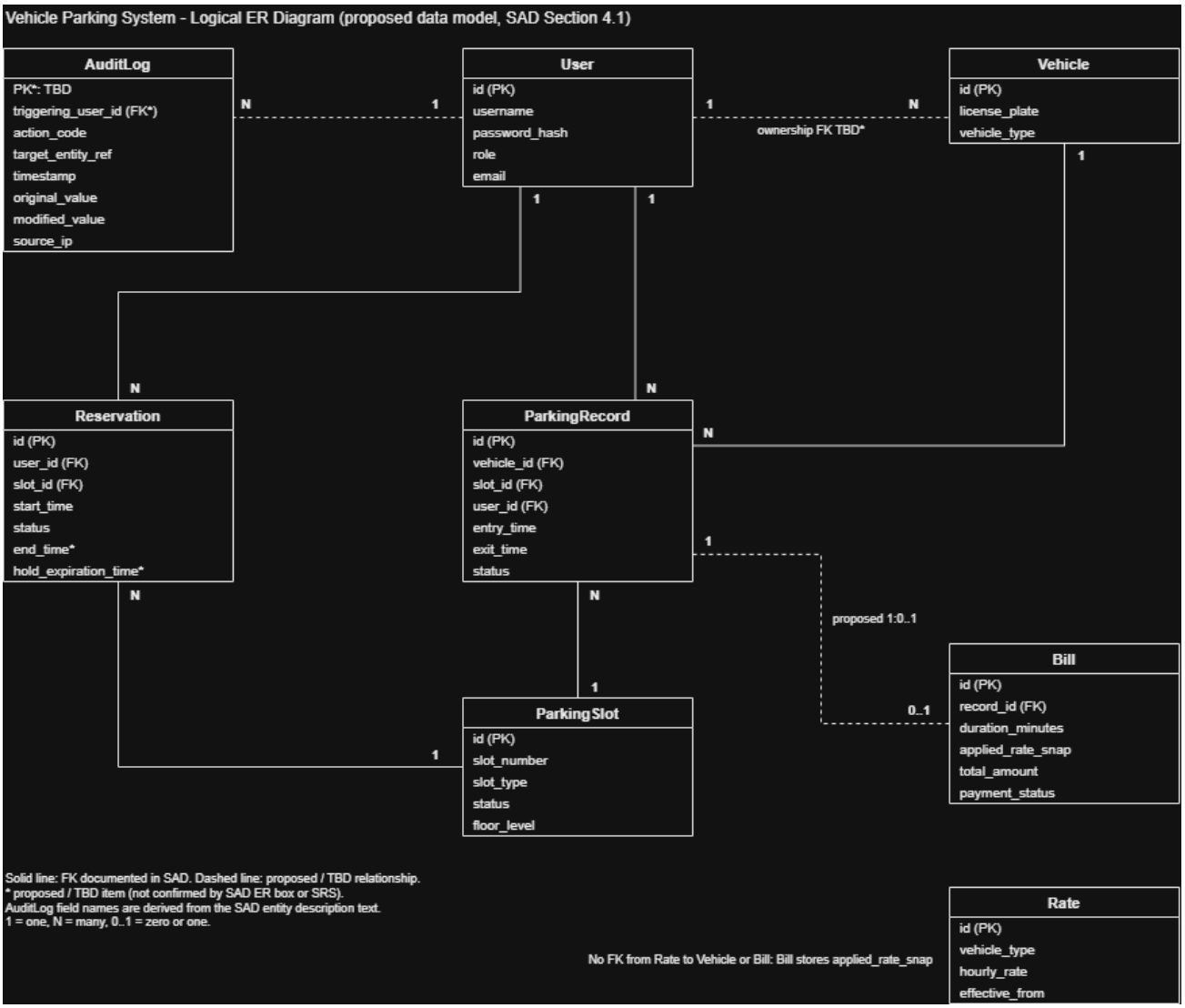
4.1 Design Overview & Entity-Relationship Data Model

The persistence layer is structured to enforce strong referential integrity, eliminate data redundancy, and maintain historical accuracy even when operational configurations change.

Data Model Entity Definitions

1. **User (`users_user`):** Stores account profiles, hashed credentials, contact information, and assigned system roles (`ADMIN`, `ATTENDANT`, `CUSTOMER`).
2. **Vehicle (`vehicles_vehicle`):** Contains physical vehicle information, primary license plate string, vehicle category enum (`TWO_WHEELER`, `COMPACT`, `SUV`, `EV`), and owner references.
3. **ParkingSlot (`slots_parkingslot`):** Represents an individual physical bay. Features include slot number code, floor level, assigned vehicle category compatibility, status enum (`AVAILABLE`, `OCCUPIED`, `RESERVED`, `MAINTENANCE`), and active status flag.
4. **ParkingRecord (`operations_parkingrecord`):** Transactional table tracking vehicle visits. Stores check-in timestamp, check-out timestamp, current status (`ACTIVE`, `COMPLETED`), and foreign key references to `Vehicle`, `ParkingSlot`, and processing `User`.
5. **Rate (`billing_rate`):** Configuration schema defining pricing strategies. Contains vehicle category, base hourly fee, initial grace period minutes, overnight surcharge rates, and effective date range flags.
6. **Bill (`billing_bill`):** Inalterable financial invoice created upon check-out. Contains link to `ParkingRecord`, calculated total duration in minutes, applied tariff rate snapshot, base amount, tax breakdowns, final total cost, and payment status (`PENDING`, `PAID`, `WAIVED`).
7. **Reservation (`reservations_reservation`):** Tracks pre-booked slots (Target Scope). Contains user link, reserved slot link, start time, end time, hold expiration timestamp, and booking status (`CONFIRMED`, `FULFILLED`, `CANCELLED`, `EXPIRED`).
8. **AuditLog (`audit_auditlog`):** Security record capturing system events. Stores triggering user ID, action code, target entity reference, timestamp, original value state, modified value state, and source IP address.

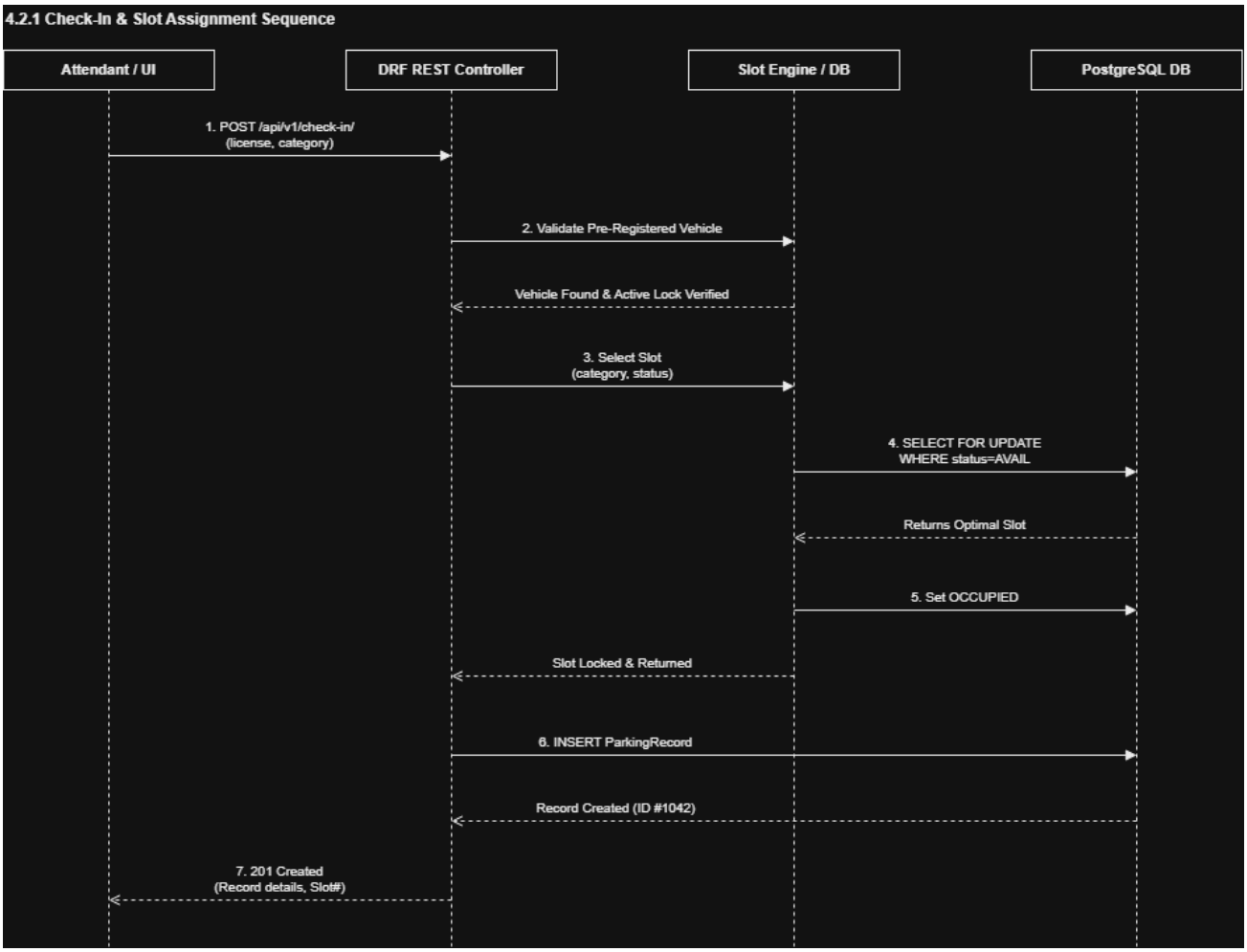
Logical Entity-Relationship Structure



4.2 Detailed Sequence Diagrams

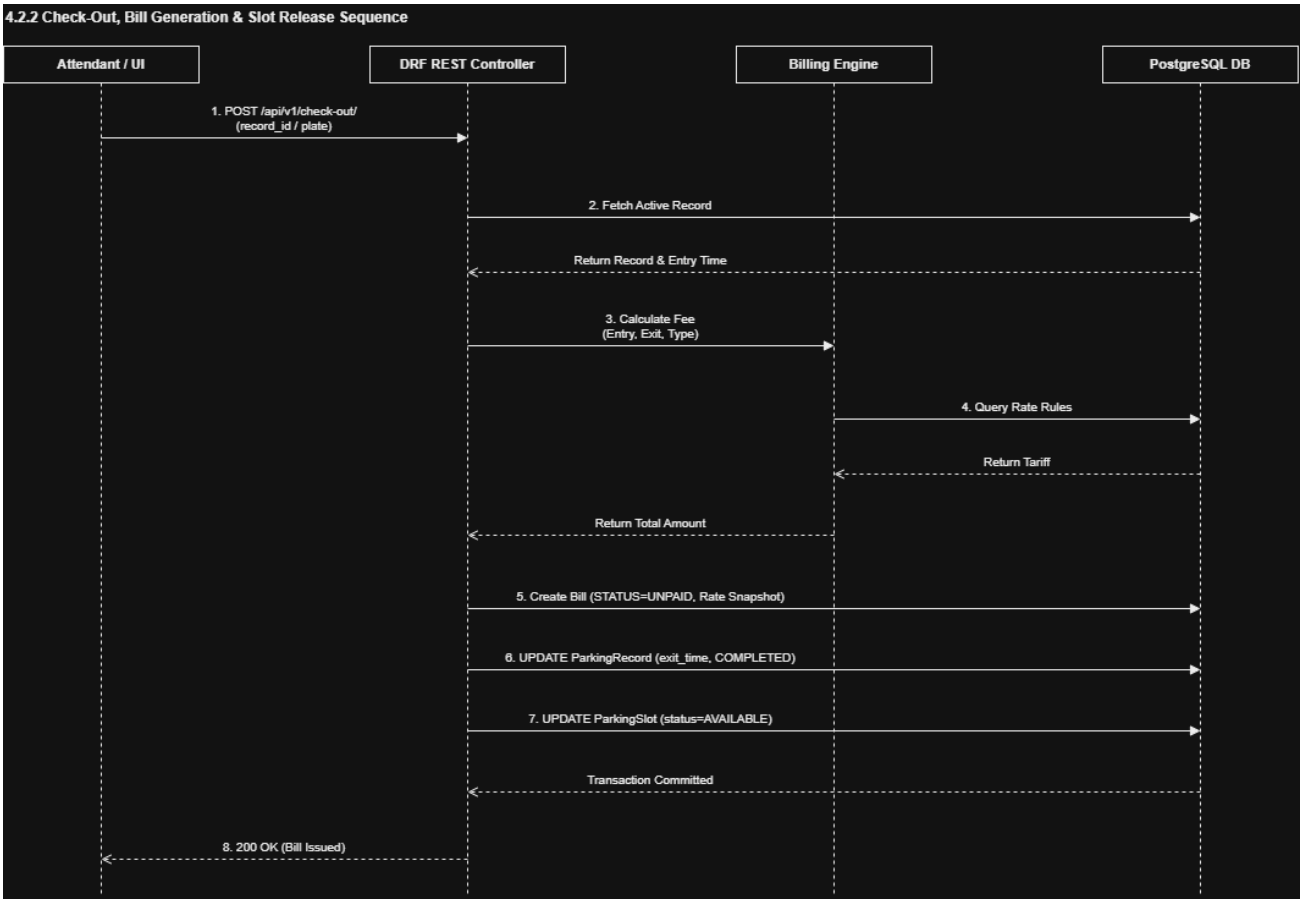
4.2.1 Check-In & Slot Assignment Sequence

The sequence below illustrates an attendant executing a vehicle check-in operation. The system verifies pre-registration, validates unique active sessions via `SELECT FOR UPDATE` row locks, verifies slot availability, updates slot status to OCCUPIED, and creates a ParkingRecord.



4.2.2 Check-Out, Bill Generation, & Slot Release Sequence

The sequence below details the exit operation: calculating parking duration, matching pricing rules, issuing an immutable bill with applied rate snapshot, clearing slot occupancy, and updating record history (decoupled from payment status).



4.3 RESTful API Design Specification

1. Vehicle Management Endpoints

- **`GET /api/v1/vehicles/`**
  - **Description:** Retrieve list of registered vehicles.
  - **Authorization:** JWT required (`Attendant`, `Admin`).
  - **Response (200 OK):**[

```
 {
   "id": 12,
   "license_plate": "KA-01-MA-1234",
   "vehicle_type": "SUV",
   "owner_name": "John Doe",
   "created_at": "2024-11-01T10:00:00Z"
 }
]
```

- **`POST /api/v1/vehicles/`**
  - **Description:** Register a new vehicle entry in registry.
  - **Authorization:** JWT required (`Attendant`, `Admin`).
  - **Request Body:**{

    "license_plate": "KA-05-NB-5678",

    "vehicle_type": "COMPACT",

    "owner_name": "Jane Smith"

    }

  - **Response (201 Created):** Returns created vehicle object.
  - **Error Responses:** `400 Bad Request` (Invalid plate format), `409 Conflict` (Vehicle exists).

2. Parking Slot Endpoints

- **`GET /api/v1/slots/`**
    - **Description:** Query all parking slots and live status map.
    - **Authorization:** Public or Authenticated users.
    - **Query Parameters:** `?floor=1&status=AVAILABLE&type=COMPACT`
    - **Response (200 OK):**[

```
 {
   "id":  101,
   "slot_number": "A-101",
   "floor_level": 1,
   "slot_type": "COMPACT",
   "status": "AVAILABLE"
 }
]
```

3. Check-In Operational Endpoint

- **`POST /api/v1/check-in/`**
    - **Description:** Process incoming vehicle, assign slot, start session.
    - **Authorization:** JWT required (`Attendant`, `Admin`).
    - **Request Body:**{

```
 "license_plate": "KA-01-MA-1234",
 "vehicle_type": "SUV"
}
```

    - **Response (201 Created):**{

```
 "record_id": 8492,
 "license_plate": "KA-01-MA-1234",
 "assigned_slot": "B-204",
```

"entry_time": "2024-11-18T14:30:00Z",

"status": "ACTIVE"

}

- **Error Responses:** `404 Not Found` (No matching slot available), `400 Bad Request` (Vehicle already inside lot).

4. Check-Out Operational Endpoint

- **`POST /api/v1/check-out/`**
  - **Description:** Complete session, update slot to available, generate invoice bill.
  - **Authorization:** JWT required (`Attendant`, `Admin`).
  - **Request Body:**{

"record_id": 8492,

"payment_method": "CASH"

}

  - **Response (200 OK):**{

"bill_id": 3021,

"record_id": 8492,

"license_plate": "KA-01-MA-1234",

"duration_minutes": 135,

"applied_rate_per_hour": 40.00,

"base_amount": 80.00,

"tax_amount": 14.40,

"total_amount": 94.40,

"payment_status": "UNPAID"

}

5. Reservation Endpoints (Provisional Scope)

- **`POST /api/v1/reservations/`**
  - **Description:** Reserve a specific slot for a future time window.
  - **Authorization:** JWT required (`Customer`, `Admin`).
  - **Request Body:**{

    "slot_id": 101,

    "start_time": "2024-11-19T09:00:00Z",

    "end_time": "2024-11-19T12:00:00Z"

    }

6. Billing Configuration Endpoints

- **`PUT /api/v1/billing/rates/{id}/`**
  - **Description:** Update pricing structure rules.
  - **Authorization:** JWT required (`Admin` only).

4.4 Error Handling, Logging, & Monitoring

7. Reporting, Audit & Notification Endpoints

- **`GET /api/v1/reports/occupancy/`**
  - **Description:** Retrieve real-time and historical occupancy metrics.
  - **Authorization:** JWT required (`Admin`).
- **`GET /api/v1/audit/logs/`**
  - **Description:** Query system audit trails and administrative overrides.
  - **Authorization:** JWT required (`Admin`).
- **`POST /api/v1/notifications/send/`**
  - **Description:** Dispatch notification alerts to target users (Provisional).
  - **Authorization:** JWT required (`Admin`).

Standardized JSON Error Response Format

All API components emit standardized error structures to simplify client processing:{

"error": {

"code": "SLOT_UNAVAILABLE",

```
    "message": "No available slot matching vehicle category 'SUV' on any floor.",
    "timestamp": "2024-11-18T14:31:02Z",
    "details": []
  }
}
```

System Log Levels

- **`DEBUG`:** Internal execution traces, raw SQL queries during development.
- **`INFO`:** Key business events (Successful check-in, completed check-out, login success).
- **`WARNING`:** Unsuccessful authentication attempts, slot capacity reaching >90%.
- **`ERROR`:** Failed transactions, database timeouts, external API payload mismatch.
- **`CRITICAL`:** Database connectivity failures, unhandled runtime crashes requiring intervention.

4.5 User Experience (UX) Design Guidelines

1. **Attendant High-Speed Workspace:** The primary operations interface is optimized for rapid keyboard-driven input. Check-in operations require minimum form fields (License Plate input with auto-suggest, vehicle type dropdown) to complete under 3 seconds.
2. **Visual Status Grid Map:** Color-coded slot representation:
    - **Green:** Available
    - **Red:** Occupied
    - **Yellow:** Reserved
    - **Gray:** Out of Service / Maintenance
3. **Responsive Layout:** Adapts seamlessly between desktop workstations (Attendant booths), tablets, and mobile web browsers.

4.6 Open Issues, Technical Gaps, & Implementation Roadmap

- **SRS v1.0 vs v1.1 Version Discrepancy & Requirement Verification**
- *Issue:* Discrepancy between authoritative SRS v1.0 baseline (FR-01 to FR-14) and provisional team backlog requirements (FR-15 to FR-30). Unapproved enterprise capabilities have been removed from committed scope pending faculty baseline approval.

- *Resolution:* The proposed architecture restricts baseline scope to FR-01 through FR-14. FR-15 through FR-30 are retained purely as provisional design structures subject to pending verification.

BR-12 Verification (Business Rule Compliance)

- *Issue:* Verification of BR-12 business rule compliance (overnight grace periods, rate calculations across day boundaries) requires pending validation against exact billing formulas.
- *Action:* Test case design pending verification during Phase 2 development.

Security NFR-SEC-06 & NFR-SEC-07 Implementation

- *Issue:* Full specification of security controls NFR-SEC-06 (PII field encryption at rest) and NFR-SEC-07 (audit trail non-repudiation) requires pending implementation details.
- *Action:* Designated as proposed features for security hardening review.

---

5. Appendices

5.1 Glossary

- **ANPR:** Automatic Number Plate Recognition technology utilizing optical character recognition.
- **API:** Application Programming Interface.
- **DRF:** Django REST Framework.
- **JWT:** JSON Web Token used for stateless user session authentication.
- **Modular Monolith:** Software architecture combining clean module isolation with monolithic deployment simplicity.
- **RBAC:** Role-Based Access Control model.
- **SPA:** Single Page Application frontend pattern.

5.2 References

1. *Software Requirements Specification (SRS) for Vehicle Parking System*, Version 1.1.
2. *Django REST Framework Documentation*, https://www.django-rest-framework.org/
3. *React 18 Documentation*, https://react.dev/
4. *OWASP Top 10 Web Application Security Risks*, 2021.
5. *IEEE Std 1016-2009 System Design Specifications Standard*.

5.3 Tools & Development Technologies

- **Development Environments:** VS Code, PyCharm, Postman API Client.
- **Modeling Tools:** PlantUML, Draw.io, Lucidchart.
- **Version Control:** Git, GitHub Organization Workspace.
- **Containerization:** Docker, Docker Compose for local development stack orchestration.